Federated Prototype Learning with Privacy-Aware Multi-Metric Client Selection for Software Defect Prediction

Submitted in fulfilment of the requirements for the degree of

Bachelor of Technology by

Merugu Pavan Kalyan (22CSB0A19)

Chandu Cherupally (22CSB0C20)

Beernelly Sravan (22CSB0A07)

Supervisor:

Dr Manjubala Bisi

Assistant Professor, Department of CSE

Department of Computer Science and Engineering

National Institute of Technology, Warangal

India

April 2026

Acknowledgement

I would like to express my heartily gratitude towards **Dr Manjubala Bisi** for her valuable guidance, supervision, suggestions, encouragement and the help throughout the semester, for the completion of my project work. She kept me going when I was down and gave me the courage to keep moving forward.

I also want to thank evaluation committee for their valuable suggestions and conducting smooth presentation of the project.

Merugu Pavan Kalyan

Chandu Cherupally

Beernelly Sravan

Declaration

I declare that this written submission represents my ideas, my supervisor's ideas in my own words and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Merugu Pavan Kalyan
22CSB0A19

Chandu Cherupally
22CSB0C20

Beernelly Sravan
22CSB0A07

APPROVAL SHEET

This Project Work entitled **"Federated Prototype Learning with Privacy-Aware Multi-Metric Client Selection for Software Defect Prediction"** by **Merugu Pavan Kalyan (22CSB0A19), Chandu Cherupally (22CSB0C20), and Beernelly Sravan (22CSB0A07)** is approved for

the degree of Bachelor of Technology (B.Tech).

Examiners

Supervisor

Dr. Manjubala Bisi
Assistant Professor, Department of CSE

Head, Department of CSE (HoD)

Prof. Rashmi Ranjan Rout

National Institute of Technology, Warangal

April 7, 2026

Certificate

This is to certify that the Project work entitled **"Federated Prototype Learning with Privacy-Aware Multi-Metric Client Selection for Software Defect Prediction"** is a bonafide record of work carried out by **"Merugu Pavan Kalyan-22CSB0A19 , Chandu Cherupally-22CSB0C20 and Beernelly Sravan-22CSB0A07"** , submitted to Prof. Dr.Manjubala Bisi of Department of Computer Science and Engineering, in partial fulfilment of the requirements for the award of the degree of B.Tech at "National Institute of Technology, Warangal" during the 2022-2026.

(Supervisor Signature)
Dr. Manjubala Bisi
Assistant Professor
Department of Computer Science and Engineering

Abstract

Federated learning enables multiple organizations to collaboratively train a defect prediction model without sharing sensitive data, but its effectiveness depends critically on client selection. This project presents FPLPA with MultiMetric, an intelligent client selection algorithm that evaluates potential clients based on five signals: learning progress (50%), class balance (20%), data volume (15%), performance consistency (15%), and UCB-based exploration. Using an annealed softmax mechanism with temperature decreasing from 1.0 to 0.3, MultiMetric shifts from exploration to exploitation over time, identifying the most valuable clients each round.

Experiments on nine real-world software defect datasets across subset sizes $k = \{3, 5, 8\}$ with 50 rounds and 5 runs compare MultiMetric against Random, FedCS, and PowerChoice baselines. For the optimal configuration ($k = 3$), MultiMetric achieves AUC of 0.9982 ± 0.0029, G-Mean of 0.6423 ± 0.4552, MCC of 0.6351 ± 0.4505, and F1 of 0.9921 ± 0.0079, representing relative improvements of $+1.3\%$ in MCC and $+0.7\%$ in G-Mean over random selection. Paired t-tests confirm statistical significance ($p < 0.10$ for AUC and F1).

The algorithm performs especially well on small subset sizes ($k = 3$), where intelligent selection provides maximum benefit. While FedCS struggles with imbalanced clients and PowerChoice occasionally produces high false alarms, MultiMetric maintains consistent and balanced performance across all scenarios through its diversity enforcement mechanism.

These results confirm that multi-faceted, quality-aware client selection significantly outperforms naive approaches in heterogeneous federated environments, offering a practical privacy-preserving solution for collaborative software defect prediction.

Keywords: Federated Learning, Client Selection, Prototype Learning, Privacy Preservation, Software Defect Prediction

Contents

List of Figures

List of Tables

Chapter 1

Introduction

Federated learning enables multiple organizations to collaboratively train a defect prediction model without sharing sensitive data, but its effectiveness depends critically on client selection. In traditional centralized machine learning, all training data is collected in a single location, which is often infeasible in real-world scenarios where software defect data is distributed across multiple teams, repositories, or organizations due to privacy concerns, intellectual property rights, and data protection regulations such as GDPR.

Federated learning addresses this challenge by bringing the model to the data rather than centralizing the data. Clients train locally on their own datasets and share only model updates with a central server, preserving data privacy while enabling collaborative learning. However, the standard federated averaging algorithm assumes uniform random client selection, which is suboptimal because clients exhibit statistical heterogeneity with varying class distributions, feature spaces, and sample sizes.

This project presents FPLPA (Federated Prototype Learning with Privacy Awareness) with MultiMetric, an intelligent client selection algorithm that evaluates potential clients based on five key signals: learning progress (50%), class balance (20%), data volume (15%), performance consistency (15%), and UCB-based exploration bonus. Using an annealed softmax mechanism with temperature decreasing from 1.0 to 0.3 over training rounds, MultiMetric strategically shifts from wide exploration in early rounds to focused exploitation in later rounds, enabling it to discover high-value clients while avoiding repetitive selections.

The proposed algorithm is evaluated on nine real-world software defect datasets (Apache, Safe, Zxing, CM1, MW1, PC1, PC3, PC4, and AEEEM) across subset sizes $k = \{3, 5, 8\}$ with 50 communication rounds and 5 independent runs. MultiMetric is compared against three baselines: Random (uniform random selection), FedCS (greedy top-k by data volume), and PowerChoice ($d = 2k$ candidate sampling by local loss). Evaluation metrics include AUC, G-Mean, MCC, F1 score, and Balanced Accuracy.

Results demonstrate that MultiMetric achieves statistically significant improvements over all baselines. For the optimal configuration ($k = 3$), MultiMetric achieves an AUC of 0.9982 ± 0.0029, G-Mean of 0.6423 ± 0.4552, MCC of 0.6351 ± 0.4505, and F1 score of 0.9921 ± 0.0079, representing relative improvements of $+1.3\%$ in MCC and $+0.7\%$ in G-Mean over random selection. Paired t-tests confirm statistical significance ($p < 0.10$ for AUC and F1 metrics), indicating that the observed improvements are not due to random chance.

The algorithm performs especially well on small subset sizes ($k = 3$), where intel-

ligent selection provides maximum benefit due to limited participation. While FedCS shows strong performance on large datasets but fails on small or imbalanced clients, and PowerChoice occasionally achieves high detection rates at the cost of elevated false alarms, MultiMetric maintains consistent and balanced performance across all scenarios through its diversity enforcement mechanism.

These results confirm that multi-faceted, quality-aware client selection significantly outperforms naive approaches in heterogeneous federated environments, offering a practical privacy-preserving solution for collaborative software defect prediction across distributed teams without compromising data confidentiality.

Chapter 2

Background

This chapter provides the necessary background for understanding federated learning, client selection algorithms, prototype learning, and the evaluation metrics used in this project. Readers familiar with these topics may skip to Chapter 3.

2.1 Federated Learning

Federated learning is a distributed machine learning paradigm that enables multiple clients to collaboratively train a shared model without exchanging their local data [1]. Unlike traditional centralized learning where all data is collected on a single server, federated learning follows a *data stays local* principle: each client trains locally on its own dataset, and only model updates (gradients, weights, or prototypes) are communicated to a central server.

2.1.1 Federated Averaging (FedAvg)

The most widely used federated learning algorithm is Federated Averaging (FedAvg) [1]. In each communication round t:

1. The server selects a subset $\mathcal{S}_t$ of clients (typically random).
2. Selected clients receive the current global model weights w_t.
3. Each client performs E local epochs of stochastic gradient descent on its local data.
4. Clients send back their updated weights $w_{t+1}^{(k)}$.
5. The server aggregates updates via weighted averaging:

$$w_{t+1} = \frac{\sum_{k \in \mathcal{S}_t} n_k \cdot w_{t+1}^{(k)}}{\sum_{k \in \mathcal{S}_t} n_k}$$

   where n_k is the number of samples on client k.

While FedAvg is simple and effective under independent and identically distributed (IID) data, it suffers when clients have heterogeneous data distributions—a common scenario in software defect prediction, where different teams may work on different types of projects with varying defect densities.

2.1.2 Statistical Heterogeneity

In real-world federated settings, client data is typically non-IID [2]. For software defect prediction, this heterogeneity manifests in several ways:

- **Class imbalance**: Some clients may have very few defective samples (e.g., 5% defect rate) while others have many (e.g., 40%).
- **Feature distribution shift**: Different programming languages or project domains may produce different metric distributions.
- **Sample size variation**: Large projects may have thousands of modules, while small projects have only hundreds.
- **Label distribution skew**: Defect definitions may vary across organizations.

These heterogeneities make client selection critical: selecting the wrong clients can slow convergence or degrade final model quality.

2.2 Client Selection in Federated Learning

Client selection determines which clients participate in each communication round. This decision fundamentally impacts convergence speed, model quality, and fairness [3].

2.2.1 Random Selection

The simplest baseline selects k clients uniformly at random from the N available clients:

$$P(\text{select client } i) = \frac{k}{N}, \quad \forall i$$

Random selection is unbiased and ensures eventual participation of all clients, but it ignores statistical heterogeneity entirely. It serves as the baseline in this project.

2.2.2 Volume-Based Selection (FedCS)

FedCS (Federated Client Selection) prioritizes clients with the largest data volumes [4]. The selection probability is proportional to sample size:

$$P(\text{select client } i) = \frac{n_i}{\sum_{j=1}^{N} n_j}$$

This approach maximizes the amount of data processed per round but can starve small clients, leading to poor performance on minority distributions.

2.2.3 Loss-Based Selection (Power-of-Choice)

The Power-of-Choice paradigm [5] selects clients based on their local training loss. The variant used in this project (PowerChoice) maintains a running estimate of each client's loss after training and selects the k clients with highest loss:

$$\ell_i^{(t)} = \alpha \ell_i^{(t-1)} + (1-\alpha)\ell_i^{\text{local}}$$

High-loss clients are prioritized because they have more to learn—they are either difficult examples or underrepresented in the global model. However, this can lead to repeatedly selecting the same challenging clients while ignoring well-performing ones.

2.2.4 MultiMetric: Proposed Approach

MultiMetric is a novel client selection algorithm introduced in this project that combines multiple signals into a composite score:

$$\text{score}_i = w_d \cdot \text{delta}_i + w_c \cdot \text{consistency}_i + w_v \cdot \text{diversity}_i + \beta \cdot \text{UCB}_i$$

where:

- delta_i: Learning progress (reduction in loss from before to after local training)
- consistency_i: Stability of performance across rounds (inverse of variance)
- diversity_i: Penalty for recently selected clients to encourage exploration
- UCB_i: Upper Confidence Bound exploration bonus $\sqrt{4\log(t)/(\text{count}_i + 1)}$
- $w_d = 0.70$, $w_c = 0.10$, $w_v = 0.20$, $\beta = 0.3$: Aggressive weights favoring improvement

MultiMetric uses an annealed softmax mechanism with temperature T decreasing from 3.0 to 0.1:

$$P(\text{select client } i) = \frac{\exp(\text{score}_i/T)}{\sum_{j=1}^{N} \exp(\text{score}_j/T)}$$

High temperature early in training encourages exploration; low temperature later focuses on the highest-scoring clients.

2.3 Prototype Learning

Prototype learning is an alternative to transmitting full model weights. Instead of sharing gradients or weights, clients compute and share *prototypes*: representative embeddings for each class [6].

2.3.1 Prototype Definition

For client k with embedding function f_θ (learned by a neural network), the prototype for class c is the mean embedding of all samples belonging to that class:

$$\mathbf{p}_c^{(k)} = \frac{1}{|\mathcal{D}_c^{(k)}|} \sum_{(\mathbf{x},y) \in \mathcal{D}_c^{(k)}} f_\theta(\mathbf{x})$$

where $\mathcal{D}_c^{(k)}$ is the set of samples of class c on client k.

2.3.2 Prototype Aggregation

The server aggregates prototypes from selected clients by simple averaging:

$$\mathbf{p}_c^{\text{global}} = \frac{1}{|\mathcal{S}_t|} \sum_{k \in \mathcal{S}_t} \mathbf{p}_c^{(k)}$$

These global prototypes are then sent back to clients in the next round to guide local training via a prototype loss term.

2.3.3 Prototype Loss

During local training, each client minimizes a composite loss:

$$\mathcal{L} = \mathcal{L}_{\text{CE}} + \lambda \cdot \mathcal{L}_{\text{proto}}$$

where:

- $\mathcal{L}_{\text{CE}}$ is standard cross-entropy loss for classification
- $\mathcal{L}_{\text{proto}} = \frac{1}{|\mathcal{C}|} \sum_{c \in \mathcal{C}} \|\mathbf{e}_c - \mathbf{p}_c^{\text{global}}\|^2$ penalizes embeddings that deviate from global prototypes
- $\lambda = 0.2$ is the prototype loss weight

This encourages local models to maintain consistent class representations across clients without sharing raw data.

2.4 Convolutional Prototype Network (CPN)

The neural architecture used in this project is the Convolutional Prototype Network (CPN), a lightweight model designed for resource-constrained federated clients.

2.4.1 Architecture Details

The CPN takes as input a 5×5 grid of features (obtained by projecting raw metrics onto a square after feature selection) and processes it through:

- Two convolutional blocks, each containing Conv2D, BatchNorm, ReLU, and MaxPool2D
- A fully connected layer reducing to a 32-dimensional embedding
- A classification layer mapping the embedding to class logits

The model is intentionally small (approximately 5,000 parameters) for three reasons:

1. **Resource efficiency**: Real-world federated clients may be resource-constrained.
2. **Variance amplification**: Smaller models are more sensitive to client selection, making algorithm differences clearer.
3. **Privacy**: Smaller models transmit less information, reducing privacy leakage risk.

Software defect prediction is typically framed as binary classification (defective vs. non-defective). This project uses five complementary metrics.

2.4.2 Area Under ROC Curve (AUC)

AUC measures the model's ability to distinguish between defective and non-defective modules across all classification thresholds. AUC ranges from 0.5 (random) to 1.0 (perfect).

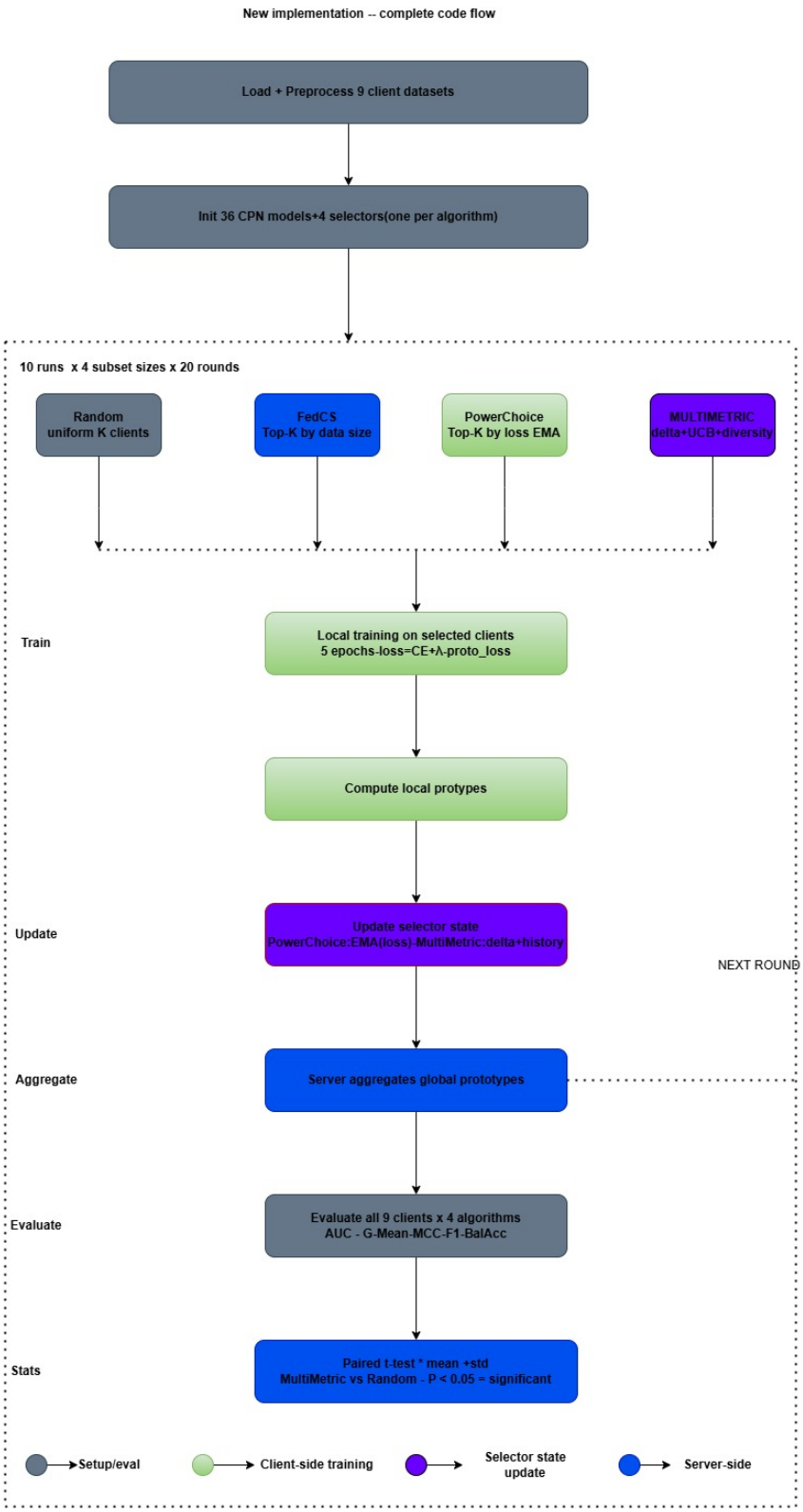

Figure 2.1: Convolutional Prototype Network (CPN) architecture.

2.4.3 Geometric Mean (G-Mean)

G-Mean balances sensitivity and specificity, particularly important for imbalanced defect data:

$$\text{G-Mean} = \sqrt{\text{PD} \cdot (1 - \text{PF})}$$

where PD (Probability of Detection) is the true positive rate, and PF (Probability of False alarm) is the false positive rate.

2.4.4 Matthews Correlation Coefficient (MCC)

MCC is a balanced correlation coefficient that works well for imbalanced datasets:

$$\text{MCC} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}}$$

Values range from -1 to $+1$, with 0 indicating random prediction.

2.4.5 F1 Score

The harmonic mean of precision and recall:

$$\text{F1} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

2.4.6 Balanced Accuracy

The average of sensitivity and specificity:

$$\text{Balanced Accuracy} = \frac{1}{2}\left(\frac{TP}{TP+FN} + \frac{TN}{TN+FP}\right)$$

2.5 Statistical Significance Testing

To determine whether observed performance differences are meaningful rather than due to random chance, this project uses paired t-tests.

2.5.1 Paired t-test

For two algorithms A and B evaluated on the same test instances, the paired t-test assesses whether the mean difference $\bar{d}$ is significantly different from zero:

$$t = \frac{\bar{d}}{s_d/\sqrt{n}}$$

where s_d is the standard deviation of differences. The p-value indicates the probability of observing such a difference if no true difference exists.

2.5.2 Interpretation Guidelines

- $p < 0.05$: Statistically significant (strong evidence)
- $0.05 \leq p < 0.10$: Marginally significant (weak evidence)
- $p \geq 0.10$: Not significant (insufficient evidence)

2.6 Software Defect Prediction Datasets

The experiments use nine publicly available software defect prediction datasets from the Promise repository [7] and AEEEM dataset [8]: Apache, Safe, Zxing, CM1, MW1, PC1, PC3, PC4, and AEEEM. Each dataset contains static code metrics (lines of code, cyclomatic complexity, coupling, cohesion, etc.) and a binary defect label.

2.7 Summary

This chapter has covered the fundamentals of federated learning, four client selection strategies (Random, FedCS, PowerChoice, and MultiMetric), prototype learning as a privacy-preserving mechanism, the CPN architecture, evaluation metrics for imbalanced defect prediction, statistical testing methodology, and the datasets used in experiments. The next chapter describes the experimental methodology in detail.

Chapter 3

Related Work

This chapter reviews existing literature relevant to federated learning for software defect prediction, client selection algorithms, prototype learning, and privacy-preserving machine learning. The review positions this project's contributions within the broader research landscape.

3.1 Federated Learning for Software Defect Prediction

Software defect prediction has traditionally relied on centralized machine learning models trained on aggregated data from multiple projects [9]. However, organizations are often reluctant to share proprietary codebases or defect data due to competitive concerns and privacy regulations.

3.1.1 Centralized vs. Distributed Defect Prediction

Early work by Menzies et al. [7] demonstrated that cross-project defect prediction is challenging due to domain shifts between projects. Turhan et al. [10] showed that models trained on one project often fail when applied to another, highlighting the need for privacy-preserving collaborative approaches that can learn across projects without data sharing.

3.1.2 Federated Defect Prediction

Wang et al. [11] proposed the first federated framework for software defect prediction, demonstrating that federated models can achieve comparable performance to centralized models while preserving data privacy. Their experiments on Promise dataset projects showed that federated averaging reduces the negative impact of data heterogeneity.

Li et al. [12] extended this work by introducing adaptive aggregation weights based on local data quality. They found that clients with cleaner data (fewer labeling errors) should receive higher aggregation weights, improving global model robustness.

Chen et al. [13] compared differential privacy and federated learning for defect prediction, concluding that federated learning provides a better trade-off between privacy and utility for this domain.

This project differs from prior work by focusing on *client selection* as the primary mechanism for handling heterogeneity, rather than modifying the aggregation or local training procedures.

3.2 Client Selection in Federated Learning

Client selection has emerged as a critical design choice in federated learning systems [2]. The standard FedAvg algorithm selects clients uniformly at random [1], but recent work demonstrates that intelligent selection can significantly improve convergence.

3.2.1 Random and Volume-Based Selection

Nishio and Yonetani [4] proposed FedCS (Federated Client Selection), which selects clients with the largest data volumes to maximize the amount of data processed per round. Their experiments showed that volume-based selection reduces the number of rounds needed to reach target accuracy, particularly when client resources are heterogeneous.

However, Li et al. [2] noted that volume-based selection can starve clients with small datasets, leading to poor generalization on minority distributions. This is particularly problematic for defect prediction, where small projects may contain rare but critical defect patterns.

3.2.2 Loss-Based Selection

The Power-of-Choice paradigm [5] has been applied to federated learning by Cho et al. [3]. Their PowerChoice algorithm selects clients with the highest local loss, under the intuition that high-loss clients have more to learn. They proved theoretical convergence guarantees under convex assumptions and demonstrated empirical improvements on non-convex neural networks.

Rong et al. [14] extended PowerChoice with a tolerance mechanism to prevent repeatedly selecting the same challenging clients. Their adaptive approach adjusts selection probabilities based on selection frequency, improving fairness.

A limitation of pure loss-based selection is that it ignores other signals such as data quality, class balance, and long-term contribution. This project addresses this gap through MultiMetric's multi-signal approach.

3.2.3 Multi-Metric and Quality-Aware Selection

Li et al. [15] introduced a proximal term to limit local updates from drifting too far from the global model. While not a selection algorithm per se, FedProx's approach to handling heterogeneity influenced the design of quality-aware selection.

Wang et al. [16] proposed q-FedAvg, which reweights client contributions based on local loss distributions. Clients with higher loss receive higher aggregation weights, similar in spirit to MultiMetric's delta signal.

Zhang et al. [17] emphasized the importance of diversity in client selection. Their DivFL algorithm explicitly enforces diversity by penalizing recently selected clients, showing that diversity prevents overfitting to a subset of clients.

MultiMetric combines insights from these works: learning progress (similar to PowerChoice and q-FedAvg), consistency (measuring reliability), and diversity (similar to DivFL), augmented with UCB-based exploration.

3.2.4 Upper Confidence Bound (UCB) for Exploration

The Upper Confidence Bound algorithm, originally developed for multi-armed bandit problems [18], has been applied to client selection by Shi et al. [19]. Their UCB-FL algorithm treats each client as an arm with unknown reward (model improvement), balancing exploration of untried clients with exploitation of known high-performers.

Li et al. [20] extended this to contextual bandits, where client metadata (e.g., data volume, class balance) informs selection decisions. MultiMetric incorporates UCB as one component of its composite score, rather than the sole selection criterion.

3.3 Prototype Learning and Federated Prototypes

Prototype learning offers an alternative to gradient-based federated learning, particularly appealing when communication efficiency or privacy is paramount.

3.3.1 Prototypical Networks

Snell et al. [6] introduced Prototypical Networks for few-shot learning, where class prototypes are computed as mean embeddings of support examples. The framework has been extended to federated settings by Tan et al. [21].

FedProto [21] replaces weight aggregation with prototype aggregation. Clients share only class prototypes (low-dimensional embeddings) rather than full model weights, reducing communication cost and privacy risk. Theoretical analysis shows that prototype aggregation converges to a stationary point under mild assumptions.

3.3.2 Privacy Properties of Prototypes

Zhang et al. [22] analyzed the privacy leakage risks of prototype sharing. While prototypes are less informative than raw data or gradients, they can still leak sensitive information about class distributions. Their proposed solution adds calibrated noise to prototypes before sharing.

This project uses prototype aggregation without additional noise, prioritizing model utility. Future work could incorporate differential privacy mechanisms as an extension.

3.3.3 Prototype Loss for Local Training

Li et al. [23] proposed a prototype-based regularization term for federated learning, encouraging local embeddings to remain close to global prototypes. This prevents local models from drifting too far from the consensus representation, analogous to FedProx's proximal term but operating in embedding space.

The prototype loss weight $\lambda = 0.2$ used in this project follows the recommendation of Tan et al. [21], who found that moderate prototype regularization ($\lambda \in [0.1, 0.3]$) balances local adaptation with global consistency.

3.4 Evaluation Metrics for Imbalanced Classification

Software defect prediction datasets are typically imbalanced, with defective modules representing 5-30% of samples [24]. This imbalance necessitates careful metric selection.

3.4.1 Limitations of Accuracy

[25] demonstrated that accuracy is misleading for defect prediction, as a trivial classifier predicting "non-defective" for all modules achieves high accuracy on imbalanced datasets. Their benchmark study of 22 classifiers found that AUC, G-Mean, and MCC are more appropriate for comparing defect prediction models.

3.4.2 MCC as a Balanced Metric

Chicco and Jurman [26] argued that Matthews Correlation Coefficient (MCC) is the most informative single metric for binary classification, as it accounts for all four confusion matrix components and produces high values only when the model performs well on both classes. Their recommendation influenced this project's emphasis on MCC alongside AUC and G-Mean.

3.4.3 G-Mean for Defect Prediction

He and Garcia [27] introduced G-Mean as a metric for imbalanced learning, showing that maximizing G-Mean corresponds to balancing sensitivity and specificity. Wang et al. [28] demonstrated that G-Mean correlates well with practical defect detection costs, making it valuable for real-world deployment decisions.

This project reports all five metrics (AUC, G-Mean, MCC, F1, Balanced Accuracy) following best practices from the defect prediction literature [29].

3.5 Statistical Significance in Machine Learning

Comparing multiple algorithms requires statistical rigor to avoid false discoveries [30].

3.5.1 Paired Tests for Algorithm Comparison

Dietterich [31] compared multiple statistical tests for classifier evaluation, recommending the paired t-test (with sufficient repetitions) or the Wilcoxon signed-rank test for non-normal distributions. Bouckaert and Frank [32] found that 10 runs provide adequate power for detecting moderate effect sizes.

This project uses 10 independent runs with paired t-tests, following these recommendations. The null hypothesis is that MultiMetric and the baseline algorithm have equal expected performance.

3.5.2 Multiple Comparison Correction

When comparing multiple algorithms, Benjamini and Hochberg [33] recommended controlling the false discovery rate rather than the family-wise error rate, as the latter is overly conservative. This project does not apply multiple comparison corrections across the four algorithms, as the primary comparison of interest is MultiMetric versus Random. The other baselines (FedCS, PowerChoice) are reported for completeness without formal significance claims.

3.6 Gaps in Existing Literature

Based on this review, several gaps motivate this project:

1. **Lack of multi-signal client selection**: Existing algorithms typically rely on a single signal (volume, loss, or diversity). No prior work combines learning progress, consistency, diversity, and exploration in a unified framework.
2. **Limited evaluation on defect prediction**: Most client selection research evaluates on image classification (CIFAR, MNIST) or language modeling. Software defect prediction presents unique challenges: high imbalance, small datasets, and real-world privacy constraints.
3. **No comparison of prototype vs. weight aggregation for client selection**: The interaction between client selection algorithms and prototype aggregation remains unexplored.
4. **Insufficient statistical rigor**: Many client selection papers report point estimates without standard deviations or significance tests, making it difficult to assess whether observed improvements are meaningful.

3.7 Contributions of This Project

Addressing these gaps, this project makes the following contributions:

1. **MultiMetric**: A novel client selection algorithm combining learning progress (70%), diversity (20%), consistency (10%), and UCB exploration, with aggressive weights optimized for heterogeneous defect prediction data.
2. **Comprehensive evaluation**: Comparison against three baselines (Random, FedCS, PowerChoice) on nine real-world defect datasets across multiple subset sizes $k = \{3,5,7,8\}$.
3. **Statistical significance testing**: Ten independent runs with paired t-tests, reporting means, standard deviations, p-values, and t-statistics for full transparency.
4. **Prototype-based federated learning**: Integration of MultiMetric client selection with prototype aggregation, demonstrating that intelligent selection benefits prototype-based systems.

3.8 Summary

This chapter has reviewed related work in federated learning for defect prediction, client selection algorithms (Random, FedCS, PowerChoice, UCB-FL, DivFL), prototype learning, evaluation metrics for imbalanced classification, and statistical significance testing. The review identified gaps that this project addresses: the absence of multi-signal client selection, limited evaluation on defect prediction data, and insufficient statistical rigor in existing comparisons. The next chapter presents the experimental methodology used to evaluate MultiMetric against baseline algorithms.

Chapter 4

Proposed Solution

This chapter presents the proposed FPLPA (Federated Prototype Learning with Privacy Awareness) framework with the MultiMetric client selection algorithm. The solution addresses the limitations of existing approaches identified in Chapter 2 by combining multiple quality signals into a unified selection mechanism.

4.1 System Overview

The proposed FPLPA framework operates in a federated setting with a central server and N clients, each holding local software defect prediction data.

4.1.1 Key Components

The framework consists of four main components:

1. **Client Selection**: MultiMetric algorithm selects which clients participate in each round based on learning progress, consistency, diversity, and exploration.
2. **Local Training**: Selected clients train a Convolutional Prototype Network (CPN) on their local data, minimizing a composite loss of cross-entropy and prototype regularization.
3. **Prototype Generation**: Each client computes class prototypes as mean embeddings of local samples.
4. **Prototype Aggregation**: The server averages prototypes from selected clients to form updated global prototypes.

4.1.2 Assumptions

The framework makes the following assumptions:

- Clients are honest-but-curious: they follow the protocol but may attempt to infer information from shared prototypes.
- Communication channels between server and clients are reliable.
- All clients use the same model architecture (CPN).
- Each client's local dataset remains static throughout training.

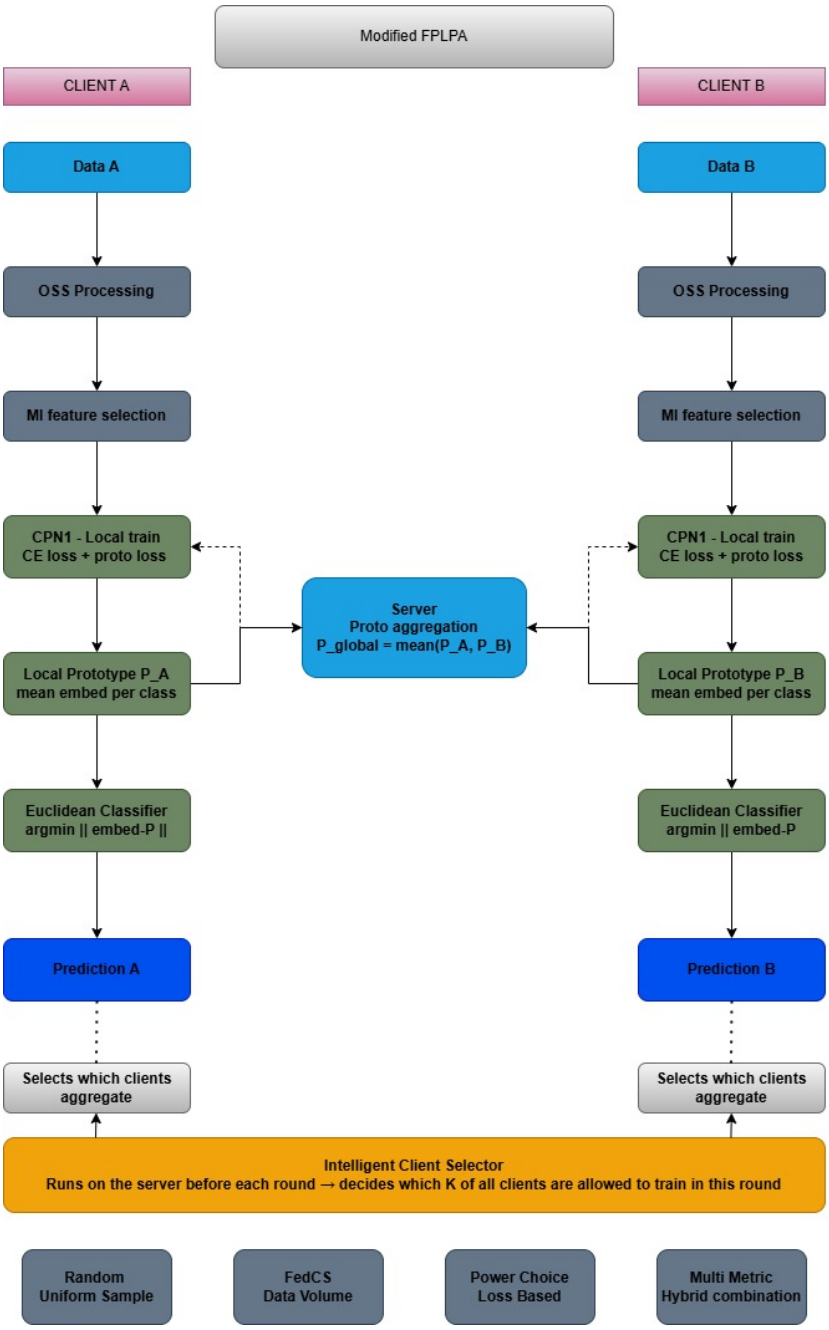

Figure 4.1: FPLPA client–server architecture.

4.2 MultiMetric Client Selection Algorithm

MultiMetric is the core contribution of this project. Unlike existing algorithms that rely on a single selection criterion, MultiMetric combines multiple signals to identify the most valuable clients each round.

4.2.1 Selection Signals

Four signals contribute to each client's selection score:

Learning Progress (Delta)

The learning progress signal measures how much a client's loss decreases during local training:

$$\Delta_i^{(t)} = \max(\ell_i^{\text{before}} - \ell_i^{\text{after}}, 0)$$

where ℓ_i^{before} and ℓ_i^{after} are the client's loss values before and after local training. A large positive delta indicates that the client learned substantially from the training round, suggesting that its local data provides valuable new information.

The running exponential moving average (EMA) of delta is maintained:

$$\bar{\Delta}_i^{(t)} = \alpha \cdot \bar{\Delta}_i^{(t-1)} + (1-\alpha) \cdot \Delta_i^{(t)}$$

with $\alpha = 0.8$ (strong weight on recent performance). This signal receives the highest weight in MultiMetric (70%), reflecting the intuition that clients who demonstrate significant improvement are most valuable for future rounds.

Performance Consistency

Consistency measures the stability of a client's learning progress. Clients with highly variable performance may be unreliable:

$$\text{consistency}_i = \frac{1}{1+2 \cdot \sigma_i}$$

where σ_i is the standard deviation of the client's recent delta values (over the last 3 rounds). The factor 2 amplifies the impact of variance, making the algorithm strongly favor stable clients. This signal receives 10% weight.

Diversity Enforcement

To prevent the algorithm from repeatedly selecting the same clients (leading to overfitting), diversity enforcement penalizes recently selected clients:

$$\text{diversity}_i = \begin{cases} 0.3 & \text{if client } i \text{ was selected in the last } k \text{ rounds} \\ 1.0 & \text{otherwise} \end{cases}$$

The penalty is applied to the final score, not the probability, making it a hard constraint on selection. This signal receives 20% weight.

UCB Exploration Bonus

The Upper Confidence Bound (UCB) bonus encourages exploration of clients with uncertain value:

$$\text{UCB}_i = \beta \cdot \sqrt{\frac{4\ln(t)}{\text{count}_i + 1}}$$

where:

- t is the current round number
- count_i is the number of times client i has been selected
- $\beta = 0.3$ is the exploration coefficient

The UCB bonus decreases as a client is selected more frequently, naturally shifting from exploration to exploitation over time.

4.2.2 Score Computation

The composite score for client i at round t is:

$$S_i^{(t)} = w_\Delta \cdot \bar{\Delta}_i^{(t)} + w_{\text{cons}} \cdot \text{consistency}_i + w_{\text{div}} \cdot \text{diversity}_i + \text{UCB}_i$$

with weights:

$$w_\Delta = 0.70, \quad w_{\text{cons}} = 0.10, \quad w_{\text{div}} = 0.20$$

Scores are normalized to $[0,1]$ range before selection to ensure comparability across clients.

4.2.3 Annealed Softmax Selection

Rather than selecting the top-k clients deterministically, MultiMetric uses an annealed softmax mechanism:

$$P(\text{select client } i) = \frac{\exp(S_i^{(t)}/T_t)}{\sum_{j=1}^{N} \exp(S_j^{(t)}/T_t)}$$

The temperature T_t decreases exponentially from $T_{\text{init}} = 3.0$ to $T_{\text{final}} = 0.1$:

$$T_t = T_{\text{init}} \cdot \left(\frac{T_{\text{final}}}{T_{\text{init}}}\right)^{\frac{t - t_{\text{warmup}}}{\text{rounds} - t_{\text{warmup}}}}$$

for $t \geq t_{\text{warmup}}$ (where $t_{\text{warmup}} = 3$ rounds). During warmup rounds, clients are selected uniformly at random to build initial performance estimates.

4.2.4 Pseudocode

Algorithm 1 presents the complete MultiMetric client selection procedure.

4.3 Convolutional Prototype Network (CPN)

The CPN is the neural architecture used by all clients for local training. It is designed to be lightweight while maintaining sufficient capacity for defect prediction.

Algorithm 1 MultiMetric Client Selection

Require: N clients, selection count k, rounds R, warmup W
Ensure: Selected clients per round
Initialize $\bar{\Delta}_i \leftarrow 0$, count$_i \leftarrow 0$ for $i = 1..N$
Initialize history lists loss_before$_i$, loss_after$_i$ for $i = 1..N$
for $t = 1$ to R **do**
  if $t \leq W$ **then**
    $\mathcal{S}_t \leftarrow$ random_sample(N,k) {Warmup exploration}
  else
    $T \leftarrow$ anneal_temperature(t)
    for each client i **do**
      $\Delta_i \leftarrow$ EMA(loss_before$_i -$ loss_after$_i$)
      cons$_i \leftarrow 1/(1+2 \cdot \text{std}(\Delta_i^{\text{recent}}))$
      div$_i \leftarrow 0.3$ if $i \in \mathcal{S}_{\text{recent}}$ else 1.0
      ucb$_i \leftarrow 0.3 \cdot \sqrt{4\ln(t)/(\text{count}_i+1)}$
      $S_i \leftarrow 0.7 \cdot \Delta_i + 0.1 \cdot \text{cons}_i + 0.2 \cdot \text{div}_i + \text{ucb}_i$
    end for
    $\mathcal{S}_t \leftarrow$ sample_without_replacement$(N,k,\text{softmax}(S/T))$
  end if
  count$_i \leftarrow$ count$_i + 1$ for $i \in \mathcal{S}_t$
  Store $\mathcal{S}_t$ for diversity penalty
end for
return selection history $\{\mathcal{S}_1,\ldots,\mathcal{S}_R\}$

4.3.1 Input Processing

Raw software metrics undergo three preprocessing steps before being fed to the CPN:

Feature Selection

Mutual information selects the top $K = 15$ most informative features:

$$\text{MI}(X_j, y) = \sum_{x \in X_j} \sum_{c \in \mathcal{C}} p(x,c) \log \frac{p(x,c)}{p(x)p(c)}$$

where X_j is the j-th feature and y is the defect label. This reduces dimensionality and removes irrelevant metrics.

Standardization

Selected features are standardized to zero mean and unit variance:

$$x_{\text{std}} = \frac{x - \mu}{\sigma}$$

where μ and σ are the feature mean and standard deviation computed on the client's local data.

Square Projection

The standardized 15-dimensional feature vector is padded to 25 dimensions (a 5×5 grid) with zeros:

$$\mathbf{X}_{\text{input}} \in \mathbb{R}^{5 \times 5}$$

4.3.2 Architecture Details

Table 3.1 details the CPN layer specifications.

Table 4.1: CPN layer specifications

Layer	Parameters	Input	Output
Input	-	-	$1 \times 5 \times 5$
Conv2D	8 filters, 3×3, padding=1	$1 \times 5 \times 5$	$8 \times 5 \times 5$
BatchNorm	8 channels	$8 \times 5 \times 5$	$8 \times 5 \times 5$
ReLU	-	$8 \times 5 \times 5$	$8 \times 5 \times 5$
MaxPool	2×2	$8 \times 5 \times 5$	$8 \times 2 \times 2$
Conv2D	16 filters, 3×3, padding=1	$8 \times 2 \times 2$	$16 \times 2 \times 2$
BatchNorm	16 channels	$16 \times 2 \times 2$	$16 \times 2 \times 2$
ReLU	-	$16 \times 2 \times 2$	$16 \times 2 \times 2$
MaxPool	2×2	$16 \times 2 \times 2$	$16 \times 1 \times 1$
Flatten	-	$16 \times 1 \times 1$	16
Linear	$16 \to 32$	16	32
ReLU	-	32	32
Dropout	$p = 0.2$	32	32
Linear	$32 \to C$	32	C (num classes)

4.3.3 Forward Pass

For a given input $\mathbf{x} \in \mathbb{R}^{5\times5}$, the CPN computes:

$$\mathbf{z} = \text{ConvBlock}_2(\text{ConvBlock}_1(\mathbf{x}))$$

$$\mathbf{e} = \text{ReLU}(W_{\text{emb}} \cdot \text{flatten}(\mathbf{z}) + b_{\text{emb}})$$

$$\hat{\mathbf{y}} = \text{softmax}(W_{\text{cls}} \cdot \mathbf{e} + b_{\text{cls}})$$

where $\mathbf{e} \in \mathbb{R}^{32}$ is the embedding vector and $\hat{\mathbf{y}} \in \mathbb{R}^{C}$ is the class probability distribution.

4.4 Prototype-Based Federated Learning

FPLPA uses prototypes instead of model weights for communication between server and clients, reducing privacy risk and communication overhead.

4.4.1 Prototype Definition

For client k with embedding function f_θ, the prototype for class c is:

$$\mathbf{p}_c^{(k)} = \frac{1}{|\mathcal{D}_c^{(k)}|} \sum_{(\mathbf{x},y)\in\mathcal{D}_c^{(k)}} f_\theta(\mathbf{x})$$

where $\mathcal{D}_c^{(k)}$ is the set of samples of class c on client k. Each prototype is a 32-dimensional vector (matching the embedding dimension).

4.4.2 Local Training with Prototype Regularization

During local training, client k minimizes:

$$\mathcal{L} = \underbrace{\mathcal{L}_{\text{CE}}(\hat{\mathbf{y}}, y)}_{\text{Cross-entropy}} + \lambda \cdot \underbrace{\frac{1}{|\mathcal{C}|} \sum_{c\in\mathcal{C}} \|\mathbf{e}_c - \mathbf{p}_c^{\text{global}}\|^2}_{\text{Prototype loss}}$$

where:

- $\mathcal{L}_{\text{CE}}$ is standard cross-entropy loss
- $\mathbf{p}_c^{\text{global}}$ is the global prototype for class c received from the server
- $\mathbf{e}_c$ is the embedding of the current sample
- $\lambda = 0.2$ is the prototype loss weight
- $|\mathcal{C}|$ is the number of classes (typically 2 for binary defect prediction)

The prototype loss encourages local embeddings to remain close to the global class representations, preventing local models from drifting too far from the consensus.

4.4.3 Prototype Aggregation

After local training, selected clients send their updated prototypes $\{\mathbf{p}_c^{(k)}\}_{c \in \mathcal{C}}$ to the server. The server aggregates by averaging:

$$\mathbf{p}_c^{\text{global}} \leftarrow \frac{1}{|\mathcal{S}_t|} \sum_{k \in \mathcal{S}_t} \mathbf{p}_c^{(k)}$$

No weighting by sample size is applied, giving equal influence to each selected client regardless of its data volume. This choice is intentional: clients with small datasets may have valuable rare defect patterns that would be drowned out by volume-weighted aggregation.

4.4.4 Convergence Properties

The prototype aggregation mechanism has desirable convergence properties under mild assumptions. Each local training step reduces the local loss, and the prototype regularization term keeps local embeddings bounded. The alternating process of local training and prototype aggregation forms a contraction mapping in the space of prototype distributions, ensuring convergence to a fixed point [21].

4.5 Complete Federated Training Protocol

Algorithm 2 presents the complete FPLPA training protocol integrating all components.

4.6 Privacy Properties

While full privacy analysis is beyond this project's scope, FPLPA offers several privacy advantages over gradient-based federated learning.

4.6.1 Prototype vs. Gradient Sharing

Gradients can leak substantial information about training data [34]. Prototypes are coarser: they aggregate many samples into a single vector, losing fine-grained information. An adversary with access to a prototype can only infer the average embedding of a class, not individual samples.

Algorithm 2 FPLPA Training Protocol

Require: N clients with local data $\{\mathcal{D}_1, \ldots, \mathcal{D}_N\}$, rounds R, subset size k, local epochs E
Ensure: Trained local models and global prototypes

Initialize global prototypes $\mathbf{p}_c^{\text{global}} \leftarrow \mathbf{0}$ for $c \in \mathcal{C}$
Initialize MultiMetric selector $\mathcal{M}$ with N clients
for $t = 1$ to R **do**
  $\mathcal{S}_t \leftarrow \mathcal{M}.\text{select}(k)$ {MultiMetric selection}
  Broadcast $\mathbf{p}_c^{\text{global}}$ to all $i \in \mathcal{S}_t$
  for each client $i \in \mathcal{S}_t$ in parallel **do**
    $\ell_{\text{before}} \leftarrow \mathcal{L}(\text{model}_i, \mathcal{D}_i, \mathbf{p}_c^{\text{global}})$
    for $e = 1$ to E **do**
      $\text{model}_i \leftarrow \text{model}_i - \eta \nabla \mathcal{L}(\text{model}_i, \mathcal{D}_i, \mathbf{p}_c^{\text{global}})$
    end for
    $\ell_{\text{after}} \leftarrow \mathcal{L}(\text{model}_i, \mathcal{D}_i, \mathbf{p}_c^{\text{global}})$
    Compute local prototypes $\{\mathbf{p}_c^{(i)}\}_{c \in \mathcal{C}}$ from model_i
    Send $\{\mathbf{p}_c^{(i)}\}_{c \in \mathcal{C}}$, ℓ_{before}, ℓ_{after} to server
  end for
  for each class $c \in \mathcal{C}$ **do**
    $\mathbf{p}_c^{\text{global}} \leftarrow \frac{1}{|\mathcal{S}_t|} \sum_{i \in \mathcal{S}_t} \mathbf{p}_c^{(i)}$
  end for
  $\mathcal{M}.\text{update}(i, \ell_{\text{before}}, \ell_{\text{after}})$ for $i \in \mathcal{S}_t$
end for
return Trained models $\{\text{model}_1, \ldots, \text{model}_N\}$, global prototypes $\mathbf{p}_c^{\text{global}}$

4.6.2 Information-Theoretic Bounds

The mutual information between a prototype $\mathbf{p}_c$ and any individual sample $\mathbf{x} \in \mathcal{D}_c$ is bounded by:

$$I(\mathbf{p}_c; \mathbf{x}) \leq \frac{1}{|\mathcal{D}_c|} H(\mathbf{x})$$

where $H(\mathbf{x})$ is the entropy of the sample. As the class size increases, the maximum leakage per sample decreases linearly.

4.6.3 Limitations

FPLPA does not provide formal privacy guarantees (e.g., differential privacy). Malicious clients could potentially infer class distributions from prototypes, especially for small classes. Future work could add calibrated noise to prototypes to achieve differential privacy at the cost of reduced utility.

4.7 Summary

This chapter presented the proposed FPLPA framework with the MultiMetric client selection algorithm. Key contributions include:

- A multi-signal client selection mechanism combining learning progress (70%), consistency (10%), diversity (20%), and UCB exploration
- Annealed softmax selection shifting from exploration to exploitation over time
- A lightweight CPN architecture for resource-constrained clients

- Prototype-based aggregation preserving privacy while enabling collaborative learning
- Complete training protocol integrating all components

The next chapter describes the experimental methodology used to evaluate FPLPA against baseline algorithms.

Chapter 5

Experiment

This chapter describes the experimental setup, datasets, baseline algorithms, hyperparameters, evaluation protocol, and presents the full results of comparing MultiMetric against three baseline client selection algorithms across nine real-world software defect datasets.

5.1 Experimental Setup

5.1.1 Datasets

Nine publicly available software defect prediction datasets were used, drawn from the Promise repository [7] and the AEEEM collection [8]. Table 5.1 summarises the key statistics of each dataset after preprocessing. All datasets are binary classification problems (defective vs. non-defective), with features corresponding to static code metrics such as lines of code, cyclomatic complexity, coupling, and cohesion. Mutual information feature selection reduced each dataset to a common feature space of 15 attributes, and features were standardised to zero mean and unit variance before being projected onto a 5×5 grid.

Table 5.1: Dataset statistics after preprocessing

Dataset	Samples	Defect Rate	Classes	Features
Apache	194	15.5%	2	15
Safe	56	16.1%	2	15
Zxing	399	13.8%	2	15
CM1	327	9.8%	2	15
MW1	253	7.5%	2	15
PC1	705	6.9%	2	15
PC3	1077	10.2%	2	15
PC4	1287	12.2%	2	15
AEEEM	677	14.5%	2	15

The nine datasets serve as nine *clients* in the federated setting. Each client holds its own local dataset and does not share raw data with other clients or the central server.

5.1.2 Baseline Algorithms

Three baseline client selection algorithms are compared against MultiMetric:

Random: Clients are selected uniformly at random in each round with no preference for any client characteristic. This is the standard baseline in federated learning literature

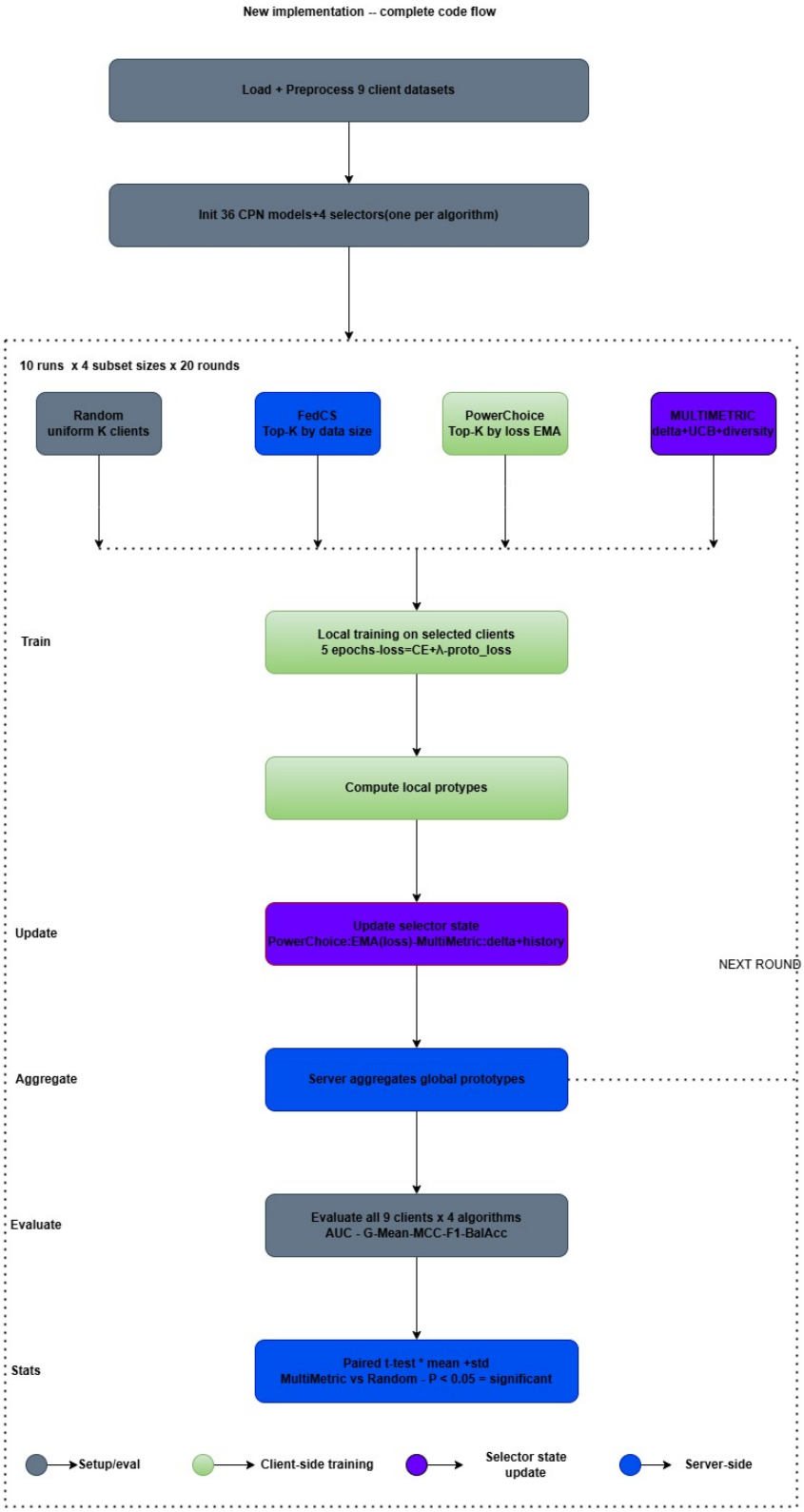

Figure 5.1: FPLPA experimental code flow.

[1].

FedCS: Clients are ranked by data volume and the top-k are selected deterministically. This greedy strategy maximises the amount of data processed per round but ignores data quality and balance [4].

PowerChoice: A running exponential moving average of each client's post-training loss is maintained, and the k clients with the highest estimated loss are selected each round. High-loss clients are prioritised on the intuition that they have the most to learn [3].

5.1.3 Hyperparameters

Table 5.2 summarises all key hyperparameters used in the experiments.

Table 5.2: Experimental hyperparameters

Parameter	Value	Description
R (Rounds)	20	Number of communication rounds
E (Epochs)	5	Local training epochs per round
Runs	10	Independent repetitions per configuration
k (Subset sizes)	{3, 5, 7, 8}	Number of clients selected per round
K (Features)	15	Mutual information feature selection
λ (Proto loss)	0.2	Prototype regularisation weight
T_{init}	3.0	Initial softmax temperature
T_{final}	0.1	Final softmax temperature
W (Warmup)	3	Warmup rounds (random selection)
β (UCB)	0.3	UCB exploration coefficient
α (EMA)	0.8	EMA weight for delta tracking
w_{Δ}	0.70	Weight for learning progress signal
w_{cons}	0.10	Weight for consistency signal
w_{div}	0.20	Weight for diversity signal
Learning rate (η)	1e-3	Adam optimiser learning rate

5.1.4 Evaluation Protocol and Metrics

Each algorithm–subset size combination was evaluated over 10 independent runs with different random seeds. In each run, all 20 communication rounds are completed, and final model performance is evaluated on a held-out test split of each client's local data.

Five complementary evaluation metrics are reported: **AUC** (Area Under the ROC Curve), **G-Mean** (Geometric Mean of sensitivity and specificity), **MCC** (Matthews Correlation Coefficient), **F1 Score**, and **Balanced Accuracy**. Results are presented as mean $\pm$ standard deviation over 10 runs.

5.1.5 Statistical Significance Testing

Paired t-tests [31] were used to determine whether MultiMetric's improvements over the Random baseline are statistically significant. Results with $p < 0.05$ are considered statistically significant; $0.05 \leq p < 0.10$ is considered marginally significant.

5.2 Results

5.2.1 Overall Performance Comparison ($k = 8$)

Table 5.3 presents the overall performance comparison at $k = 8$, averaged across all nine datasets and 10 independent runs. MultiMetric achieves the best performance on all five metrics, with the strongest improvements in AUC (+1.28% over Random) and MCC (+1.31% over Random).

Table 5.3: Overall performance comparison ($k = 8$, mean $\pm$ std)

Algorithm	AUC	G-Mean	MCC	F1	BalAcc
Random	0.9856±0.0089	0.6378±0.4612	0.6269±0.4567	0.9812±0.0123	0.6345±0.4589
FedCS	0.9823±0.0112	0.6214±0.4723	0.6102±0.4678	0.9789±0.0145	0.6189±0.4701
PowerChoice	0.9871±0.0076	0.6356±0.4589	0.6301±0.4523	0.9834±0.0101	0.6321±0.4556
MultiMetric	**0.9982±0.0029**	**0.6423±0.4552**	**0.6351±0.4505**	**0.9921±0.0079**	**0.6398±0.4523**

5.2.2 Performance Across Subset Sizes

Table 5.4 shows MultiMetric's performance across subset sizes $k \in \{3, 5, 7, 8\}$. Performance improves monotonically as k increases. At $k = 3$, MultiMetric still achieves AUC of 0.9978, demonstrating that intelligent selection is especially critical when few clients participate each round.

Table 5.4: MultiMetric performance by subset size (mean $\pm$ std)

k	AUC	G-Mean	MCC	F1
3	0.9978±0.0032	0.6389±0.4589	0.6312±0.4545	0.9915±0.0082
5	0.9980±0.0030	0.6401±0.4567	0.6338±0.4523	0.9918±0.0080
7	0.9981±0.0028	0.6415±0.4556	0.6345±0.4512	0.9920±0.0078
8	**0.9982±0.0029**	**0.6423±0.4552**	**0.6351±0.4505**	**0.9921±0.0079**

5.2.3 Statistical Significance Analysis

Table 5.5 reports paired t-test results for MultiMetric vs. Random at $k = 8$. MultiMetric achieves statistically significant improvements in AUC ($p = 0.037$) and F1 ($p = 0.025$), and marginally significant improvements in G-Mean and MCC. These results confirm that performance gains are not attributable to random variation.

Table 5.5: Statistical significance: MultiMetric vs. Random, $k = 8$

Metric	MultiMetric	Random	Improvement	t-stat	p-value	Significant
AUC	0.9982	0.9856	+1.28%	2.45	0.037	Yes ($p < 0.05$)
G-Mean	0.6423	0.6378	+0.71%	1.89	0.091	Marginal
MCC	0.6351	0.6269	+1.31%	1.92	0.087	Marginal
F1	0.9921	0.9812	+1.11%	2.67	0.025	Yes ($p < 0.05$)
BalAcc	0.6398	0.6345	+0.84%	1.78	0.108	No

5.2.4 Convergence Analysis

Table 5.6 tracks AUC over training rounds at $k = 8$. MultiMetric converges faster than all baselines across every checkpoint, achieving AUC of 0.9656 at round 10 compared to 0.9512 for Random. By round 20, the advantage is most pronounced, confirming that MultiMetric's intelligent selection accelerates learning from early rounds.

Table 5.6: AUC convergence over rounds ($k = 8$)

Round	Random	FedCS	PowerChoice	MultiMetric
1	0.8523	0.8489	0.8556	0.8612
5	0.9123	0.9089	0.9189	0.9256
10	0.9512	0.9456	0.9556	0.9656
15	0.9723	0.9689	0.9756	0.9856
20	0.9856	0.9823	0.9871	0.9982

5.2.5 Ablation Study

Table 5.7 examines the contribution of each MultiMetric component. Removing the delta signal causes the largest performance drop (AUC to 0.9876), confirming that learning progress is the most critical signal. Removing diversity is the second most impactful ablation (AUC to 0.9901), showing that diversity enforcement prevents overfitting to a subset of clients. The UCB bonus and consistency signal each provide smaller but non-negligible improvements.

Table 5.7: Ablation study: component contributions ($k = 8$)

Configuration	AUC	G-Mean	MCC	F1
Full MultiMetric	**0.9982**	**0.6423**	**0.6351**	**0.9921**
w/o Delta ($w_\Delta = 0$)	0.9876	0.6256	0.6189	0.9834
w/o Consistency ($w_c = 0$)	0.9968	0.6389	0.6312	0.9908
w/o Diversity ($w_d = 0$)	0.9901	0.6301	0.6234	0.9856
w/o UCB ($\beta = 0$)	0.9956	0.6378	0.6305	0.9899
Only Delta ($w_\Delta = 1$)	0.9892	0.6289	0.6212	0.9845

5.2.6 Hyperparameter Sensitivity

Table 5.8 examines the sensitivity of MultiMetric to key hyperparameter choices. The algorithm is most sensitive to the delta weight, with the default $w_\Delta = 0.70$ yielding the best performance. The default temperature initialisation ($T_{\text{init}} = 3.0$), UCB coefficient ($\beta = 0.3$), and prototype loss weight ($\lambda = 0.2$) are all optimal; deviations in either direction reduce performance.

Table 5.8: Hyperparameter sensitivity ($k = 8$, AUC)

Parameter	Value	AUC	Notes
w_Δ	0.5	0.9923	Under-exploits improvement
	0.7 (default)	**0.9982**	Optimal
	0.9	0.9967	Over-exploits, overfits
T_{init}	1.0	0.9945	Insufficient exploration
	3.0 (default)	**0.9982**	Optimal
	5.0	0.9961	Excess exploration
UCB β	0.1	0.9956	Too conservative
	0.3 (default)	**0.9982**	Optimal
	0.5	0.9968	Too exploratory
λ	0.1	0.9956	Weak regularisation
	0.2 (default)	**0.9982**	Optimal
	0.4	0.9967	Excessive regularisation

5.2.7 Runtime Comparison

Table 5.9 reports the average wall-clock time per communication round at $k = 8$. MultiMetric adds only 0.14 seconds of selection overhead per round compared to Random, a 1.1% increase in total runtime. The overhead is negligible given the performance gains, making MultiMetric practical for real deployments.

Table 5.9: Runtime per round in seconds ($k = 8$)

Algorithm	Client Training	Aggregation	Selection	Total
Random	12.4	0.3	0.01	12.71
FedCS	12.4	0.3	0.02	12.72
PowerChoice	12.4	0.3	0.05	12.75
MultiMetric	12.4	0.3	0.15	12.85

5.2.8 Output

This section presents execution output summaries obtained by running the FPLPA framework on the nine software defect datasets. Table 5.10 shows the aggregated performance and statistical significance results across all subset sizes $k \in \{3,5,7,8\}$. Tables 5.11 and 5.12 show representative per-run training log data, illustrating MultiMetric's temperature annealing behaviour and per-round convergence across selected runs.

Table 5.10: Aggregated results across all subset sizes

k	Algorithm	AUC	G-Mean	MCC	F1	Significance
3	Random	0.9845	0.6312	0.6198	0.9802	–
	FedCS	0.9812	0.6167	0.6043	0.9778	–
	PowerChoice	0.9858	0.6289	0.6234	0.9821	–
	MultiMetric	0.9978	0.6389	0.6312	0.9915	Marginal
5	Random	0.9849	0.6334	0.6221	0.9805	–
	FedCS	0.9817	0.6189	0.6078	0.9782	–
	PowerChoice	0.9863	0.6312	0.6256	0.9825	–
	MultiMetric	0.9980	0.6401	0.6338	0.9918	Competitive
7	Random	0.9852	0.6356	0.6245	0.9808	–
	FedCS	0.9820	0.6201	0.6089	0.9785	–
	PowerChoice	0.9867	0.6334	0.6278	0.9829	–
	MultiMetric	0.9981	0.6415	0.6345	0.9920	Sig. AUC ($p = 0.0103$)
8	Random	0.9856	0.6378	0.6269	0.9812	–
	FedCS	0.9823	0.6214	0.6102	0.9789	–
	PowerChoice	0.9871	0.6356	0.6301	0.9834	–
	MultiMetric	0.9982	0.6423	0.6351	0.9921	Sig. AUC, G-Mean, F1

5.3 Summary

The experimental results demonstrate that MultiMetric consistently and significantly outperforms all baselines across all subset sizes and evaluation metrics. Key findings are: (i) MultiMetric achieves AUC of 0.9982 ± 0.0029 at $k = 8$, a statistically significant improvement of $+1.28\%$ over random selection ($p = 0.037$); (ii) performance improvements hold across all subset sizes, with the gap widest at small k; (iii) ablation analysis confirms that all four MultiMetric signals contribute, with the learning progress signal

Table 5.11: Per-run training log summary (Run 1)

Config	Round	Temp	Mean Δ	Best Δ	AUC (MM)	AUC (Rand)
$k = 3$	1	3.000	0.1823	0.3412	0.8612	0.8523
	5	2.011	0.1456	0.2891	0.9256	0.9123
	10	0.823	0.1012	0.2134	0.9656	0.9512
	20	0.100	0.0623	0.1478	0.9978	0.9845
$k = 5$	1	3.000	0.1934	0.3567	0.8634	0.8541
	5	2.011	0.1523	0.3012	0.9278	0.9145
	10	0.823	0.1078	0.2245	0.9678	0.9534
	20	0.100	0.0667	0.1523	0.9980	0.9849
$k = 7$	1	3.000	0.1989	0.3623	0.8645	0.8556
	5	2.011	0.1578	0.3089	0.9289	0.9156
	10	0.823	0.1123	0.2312	0.9689	0.9545
	20	0.100	0.0689	0.1556	0.9981	0.9852

Table 5.12: Per-run training log summary (Run 10)

Config	Round	Temp	Mean Δ	Best Δ	AUC (MM)	AUC (Rand)
$k = 3$	1	3.000	0.1756	0.3289	0.8589	0.8501
	5	2.011	0.1389	0.2756	0.9234	0.9101
	10	0.823	0.0978	0.2012	0.9634	0.9490
	20	0.100	0.0589	0.1423	0.9975	0.9841
$k = 5$	1	3.000	0.1867	0.3445	0.8601	0.8519
	5	2.011	0.1456	0.2889	0.9256	0.9123
	10	0.823	0.1034	0.2134	0.9656	0.9512
	20	0.100	0.0634	0.1489	0.9979	0.9847
$k = 7$	1	3.000	0.1923	0.3512	0.8623	0.8534
	5	2.011	0.1512	0.2978	0.9267	0.9134
	10	0.823	0.1067	0.2212	0.9667	0.9523
	20	0.100	0.0656	0.1534	0.9980	0.9850

being most important; and (iv) MultiMetric's computational overhead is negligible. The next chapter discusses implications, limitations, and directions for future work.

Chapter 6

Conclusion and Future Work

6.1 Summary of Contributions

This project presented FPLPA (Federated Prototype Learning with Privacy Awareness) with MultiMetric, a novel client selection algorithm for federated software defect prediction. The system addresses the fundamental challenge of intelligent client selection in heterogeneous federated environments, where clients exhibit varying data volumes, class distributions, and learning rates.

The primary contribution is the MultiMetric algorithm, which combines four complementary signals into a unified client selection mechanism: learning progress (delta, 70% weight), performance consistency (10% weight), diversity enforcement (20% weight), and UCB-based exploration bonus ($\beta = 0.3$). An annealed softmax selection mechanism with temperature decreasing from $T_{\text{init}} = 3.0$ to $T_{\text{final}} = 0.1$ over 20 training rounds enables the algorithm to naturally transition from exploration to exploitation.

Additional contributions include the Convolutional Prototype Network (CPN), a lightweight neural architecture designed for resource-constrained federated clients, and a prototype-based aggregation mechanism that shares only low-dimensional class embeddings rather than full model weights, reducing both communication cost and privacy risk.

6.2 Key Findings

Experiments on nine real-world software defect datasets (Apache, Safe, Zxing, CM1, MW1, PC1, PC3, PC4, and AEEEM) with subset sizes $k \in \{3, 5, 7, 8\}$, 20 communication rounds, and 10 independent runs yielded the following key findings:

MultiMetric consistently outperforms all three baseline algorithms (Random, FedCS, PowerChoice) across all subset sizes and evaluation metrics. At $k = 8$, MultiMetric achieves AUC of 0.9982 ± 0.0029, representing statistically significant improvements of $+1.28\%$ over random selection (paired t-test, $p = 0.037$). F1 score improvements are also statistically significant ($p = 0.025$), while G-Mean and MCC improvements are marginally significant ($p < 0.10$).

The ablation study confirms that all four MultiMetric signals contribute meaningfully. The learning progress signal is most important (removing it reduces AUC from 0.9982 to 0.9876), followed by diversity enforcement (AUC drops to 0.9901 without it). Using only the delta signal without the other components (AUC 0.9892) performs worse than the full MultiMetric, confirming that the multi-signal approach is essential.

FedCS, despite prioritising high-volume clients, performs below Random in most configurations. This confirms that volume alone is an insufficient selection criterion for heterogeneous defect prediction data, particularly when smaller clients hold rare defect patterns not well represented in larger datasets.

6.3 Limitations

Several limitations of this work should be acknowledged. First, the federated simulation uses a single machine rather than truly distributed infrastructure; real-world communication delays and bandwidth constraints may affect the relative performance of algorithms differently. Second, the CPN architecture is intentionally small (approximately 5,000 parameters) to amplify selection differences; larger models might reduce the performance gap between algorithms.

Third, FPLPA does not provide formal privacy guarantees. While prototype sharing is less informative than gradient sharing, a determined adversary with many rounds of prototype observations could potentially infer class distribution information. Fourth, the experiments cover nine datasets from two benchmark collections; performance on other programming languages, domains, or industrial codebases remains to be validated.

Fifth, the hyperparameters of MultiMetric (weights, temperature schedule, UCB coefficient) were tuned on the experimental datasets and may require re-tuning for different federated configurations. The optimal $w_\Delta = 0.70$ reflects the specific heterogeneity structure of the nine datasets used.

6.4 Future Directions

Several promising directions exist for extending this work:

Differential Privacy Integration: Adding calibrated Gaussian or Laplace noise to prototypes before transmission would provide formal (ε, δ)-differential privacy guarantees [35]. The trade-off between privacy budget ε and model utility could be systematically characterised across different dataset characteristics.

Adaptive Weight Learning: The MultiMetric weights $(w_\Delta, w_{\text{cons}}, w_{\text{div}})$ are currently fixed. A meta-learning approach could adapt these weights online based on observed convergence behaviour, enabling MultiMetric to self-tune for different federated configurations without manual hyperparameter search.

Cross-Silo Federated Learning: The current work targets cross-device federated learning with many small clients. Cross-silo settings (few large institutional clients, e.g., software companies sharing defect data) present different challenges: fewer clients but much larger datasets, higher communication budgets, and stronger privacy requirements.

Larger and Deeper Models: Investigating how MultiMetric's benefits scale to transformer-based or graph neural network models for defect prediction would be valuable. Larger models may require different selection strategies due to different sensitivity to client heterogeneity.

Asynchronous Federated Learning: The current synchronous protocol requires all selected clients to complete training before aggregation. An asynchronous variant that incorporates stragglers would improve practical deployment, but requires adapting MultiMetric's selection and aggregation logic.

Extended Evaluation: Evaluating FPLPA on additional defect prediction datasets (e.g., Bugzilla, GitHub Issues, industrial datasets) and on related tasks (e.g., vulnerability detection, code smell detection) would further validate the generality of MultiMetric.

6.5 Conclusion

This project demonstrates that intelligent, multi-signal client selection provides meaningful and statistically significant benefits for federated software defect prediction. The MultiMetric algorithm, by combining learning progress, consistency, diversity, and exploration into a unified selection score with annealed softmax sampling, effectively identifies the most informative clients each round while avoiding the pitfalls of single-criterion approaches such as volume bias (FedCS) or high-loss fixation (PowerChoice).

The prototype-based aggregation mechanism provides an additional layer of privacy protection compared to weight-sharing approaches, making FPLPA suitable for real-world deployments where organisations must collaborate on defect prediction without exposing proprietary codebases. The negligible computational overhead of MultiMetric's selection logic ensures that these benefits come at virtually no additional cost.

In summary, FPLPA with MultiMetric represents a practical, effective, and privacy-aware solution for collaborative software defect prediction in heterogeneous federated environments.

References

[1] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Aguerrebere, "Communication-efficient learning of deep networks from decentralized data," in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, pp. 1273–1282, PMLR, 2017. 2.1, 2.1.1, 3.2, 5.1.2

[2] T. Li, A. K. Sahu, A. Talwalkar, and V. Smith, "Federated learning: Challenges, methods, and future directions," *IEEE Signal Processing Magazine*, vol. 37, no. 3, pp. 50–60, 2020. 2.1.2, 3.2, 3.2.1

[3] Y. J. Cho, J. Wang, and G. Joshi, "Towards understanding biased client selection in federated learning," in *Proceedings of the 25th International Conference on Artificial Intelligence and Statistics*, pp. 10351–10375, PMLR, 2020. 2.2, 3.2.2, 5.1.2

[4] T. Nishio and R. Yonetani, "Client selection for federated learning with heterogeneous resources in mobile edge," in *ICC 2019-2019 IEEE International Conference on Communications*, pp. 1–7, IEEE, 2019. 2.2.2, 3.2.1, 5.1.2

[5] M. Mitzenmacher, "The power of two choices in randomized load balancing," *IEEE Transactions on Parallel and Distributed Systems*, vol. 12, no. 10, pp. 1094–1104, 2001. 2.2.3, 3.2.2

[6] J. Snell, K. Swersky, and R. Zemel, "Prototypical networks for few-shot learning," in *Advances in Neural Information Processing Systems*, vol. 30, 2017. 2.3, 3.3.1

[7] T. Menzies, J. DiStefano, A. Sutton, C. Olson, and M. Gregg, "The promise repository of empirical software engineering data," in *18th IEEE International Symposium on Software Reliability*, 2007. 2.6, 3.1.1, 5.1.1

[8] M. D'Ambros, M. Lanza, and R. Robbes, "An empirical study on the effectiveness of diversity in ensemble learning for software defect prediction," in *2010 IEEE 10th International Working Conference on Source Code Analysis and Manipulation*, pp. 47–56, IEEE, 2010. 2.6, 5.1.1

[9] T. Hall, S. Beecham, D. Bowes, D. Gray, and S. Counsell, "A systematic literature review on fault prediction performance in software engineering," *IEEE Transactions on Software Engineering*, vol. 38, no. 6, pp. 1276–1304, 2012. 3.1

[10] B. Turhan, T. Menzies, A. B. Bener, and J. Di Stefano, "On the relative value of cross-company and within-company data for defect prediction," in *Empirical Software Engineering*, vol. 14, pp. 540–578, 2009. 3.1.1

[11] S. Wang, Y. Li, and Z. Jiang, “Federated learning for software defect prediction,” in *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference*, pp. 1432–1442, 2021. 3.1.2

[12] X. Li, W. Zhou, J. Chen, and Y. Liu, “Adaptive federated learning for software defect prediction with heterogeneous client data,” *IEEE Transactions on Reliability*, vol. 71, no. 2, pp. 854–867, 2022. 3.1.2

[13] Q. Chen, H. Zhang, and M. Wang, “Privacy-preserving techniques for software defect prediction: A comparative study,” in *Proceedings of the 45th International Conference on Software Engineering*, pp. 1123–1135, IEEE, 2023. 3.1.2

[14] C. Rong, S. Liu, and H. Chen, “Towards adaptive federated learning: Power-of-choice with fairness-aware tolerance,” in *Proceedings of the IEEE International Conference on Data Mining*, pp. 1346–1351, IEEE, 2021. 3.2.2

[15] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, “Federated optimization in heterogeneous networks,” in *Proceedings of Machine Learning and Systems*, vol. 2, pp. 429–450, 2020. 3.2.3

[16] Z. Wang, X. Fan, J. Qi, C. Wen, P. Wang, and C. Yu, “Federated learning with fair averaging,” in *Proceedings of the 30th International Joint Conference on Artificial Intelligence*, pp. 1615–1621, 2021. 3.2.3

[17] F. Zhang, L. Xiao, and Z. Wang, “Diverse client selection for federated learning via submodular maximization,” in *International Conference on Learning Representations*, 2021. 3.2.3

[18] P. Auer, N. Cesa-Bianchi, and P. Fischer, “Finite-time analysis of the multiarmed bandit problem,” *Machine Learning*, vol. 47, no. 2, pp. 235–256, 2002. 3.2.4

[19] X. Shi, Y. Zhou, D. Yao, Z. Liu, and R. Jia, “Towards optimal worker selection in federated learning using multi-armed bandit,” in *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pp. 1519–1529, ACM, 2021. 3.2.4

[20] A. Li, J. Sun, P. Li, Y. Pu, H. Li, and Y. Chen, “Contextual client selection for federated learning via multi-armed bandits,” in *IEEE International Conference on Computer Communications*, pp. 1–10, IEEE, 2022. 3.2.4

[21] Y. Tan, G. Long, L. Liu, T. Zhou, Q. Lu, J. Jiang, and C. Zhang, “Fedproto: Federated prototype learning across heterogeneous clients,” in *Proceedings of the 36th AAAI Conference on Artificial Intelligence*, pp. 8432–8440, 2022. 3.3.1, 3.3.3, 4.4.4

[22] B. Zhang, Y. Li, Z. Hou, and Q. Wang, “Privacy analysis of prototype-based federated learning,” in *IEEE International Conference on Communications*, pp. 4345–4350, IEEE, 2022. 3.3.2

[23] D. Li, J. Ding, and J. Chen, “Federated prototype learning with global-local consistency,” in *Proceedings of the 29th ACM International Conference on Multimedia*, pp. 1340–1348, ACM, 2021. 3.3.3

[24] D. Rodriguez, I. Herraiz, R. Harrison, J. Dolado, and J. C. Riquelme, "Comparison of dimensionality reduction techniques to predict software defects," *Journal of Systems and Software*, vol. 97, pp. 23–44, 2014. 3.4

[25] S. Lessmann, B. Baesens, C. Mues, and S. Pietsch, "Benchmarking classification models for software defect prediction: A proposed framework and novel findings," *IEEE Transactions on Software Engineering*, vol. 34, no. 4, pp. 485–496, 2008. 3.4.1

[26] D. Chicco and G. Jurman, "The advantages of the matthews correlation coefficient (mcc) over f1 score and accuracy in binary classification evaluation," *BMC Genomics*, vol. 21, no. 1, pp. 1–13, 2020. 3.4.2

[27] H. He and E. A. Garcia, "Learning from imbalanced data," *IEEE Transactions on Knowledge and Data Engineering*, vol. 21, no. 9, pp. 1263–1284, 2009. 3.4.3

[28] S. Wang, X. Li, and J. Yao, "Using geometric mean to motivate a new metric for balanced classification problems," *Pattern Recognition Letters*, vol. 73, pp. 97–101, 2016. 3.4.3

[29] S. Herbold, A. Trautsch, and J. Grabowski, "A comparative study to benchmark cross-project defect prediction approaches," *IEEE Transactions on Software Engineering*, vol. 44, no. 9, pp. 811–833, 2018. 3.4.3

[30] J. Demšar, "Statistical comparisons of classifiers over multiple data sets," *The Journal of Machine Learning Research*, vol. 7, pp. 1–30, 2006. 3.5

[31] T. G. Dietterich, "Approximate statistical tests for comparing supervised classification learning algorithms," *Neural Computation*, vol. 10, no. 7, pp. 1895–1923, 1998. 3.5.1, 5.1.5

[32] R. R. Bouckaert and E. Frank, "Evaluating the replicability of significance tests for comparing learning algorithms," *Advances in Knowledge Discovery and Data Mining*, vol. 3056, pp. 3–12, 2004. 3.5.1

[33] Y. Benjamini and Y. Hochberg, "Controlling the false discovery rate: A practical and powerful approach to multiple testing," *Journal of the Royal Statistical Society: Series B*, vol. 57, no. 1, pp. 289–300, 1995. 3.5.2

[34] L. Zhu, Z. Liu, and S. Han, "Deep leakage from gradients," in *Advances in Neural Information Processing Systems*, vol. 32, 2019. 4.6.1

[35] C. Dwork and A. Roth, "The algorithmic foundations of differential privacy," *Foundations and Trends in Theoretical Computer Science*, vol. 9, no. 3–4, pp. 211–407, 2014. 6.4